SANDIP UNIVERSITY
UGC Recognised

NAAC GRADE A

PROJECT REPORT

Zero-Trust Network Access: Implemented Cloudflare Tunnels and MFA to Secure Internal Admin Consoles

Project domain: Cybersecurity / Network Security **Implementation:** Python Flask + Cloudflare Zero Trust

Abstract

This project demonstrates a layered approach to protecting an internal administrative web console. A Cloudflare Tunnel provides a secure outbound connection from the origin host without requiring a publicly exposed inbound application port. Cloudflare Access is configured as the identity-aware gate in front of the application, with an access policy that restricts use to approved identities and requires multi-factor authentication (MFA). The included Flask demonstration adds a second TOTP verification step, session controls, generic authentication errors, and a basic login-attempt throttle. The project illustrates the zero-trust principle of verifying each request rather than relying on network location alone.

1. Introduction

Traditional internal admin consoles are often protected mainly by network boundaries or VPN access. If an account or endpoint is compromised, broad network trust can create unnecessary exposure. A zero-trust design applies explicit identity checks, least-privilege access, and layered controls before a user can reach a sensitive application. This project combines a Cloudflare Tunnel, Cloudflare Access policy enforcement, and application-level MFA for an educational demonstration.

2. Objectives

- Avoid directly exposing the origin application through a public inbound port.
- Restrict the admin hostname to approved users or identity-provider groups.
- Require MFA at the Cloudflare Access layer and demonstrate TOTP at the application layer.
- Apply basic secure-session settings and generic authentication failure messages.
- Document configuration, installation, testing, and production-hardening considerations.

3. System Architecture

Administrator browser → Cloudflare edge / Access policy → encrypted Cloudflare Tunnel → localhost Flask application → password + TOTP check → protected dashboard.

The Tunnel connector initiates outbound connectivity to Cloudflare. Cloudflare Access evaluates identity and policy before forwarding traffic. The Flask origin is bound to 127.0.0.1:5000 and should not be reachable directly from the public Internet. The app's TOTP step is additional defense in depth, not a replacement for Access policy enforcement.

4. Tools and Technologies

Component	Role
Python / Flask	Implements the demonstration login flow and protected dashboard.
pyotp	Verifies RFC 6238-compatible time-based one-time passwords.
Cloudflare Tunnel (cloudflared)	Connects the private origin to Cloudflare without opening an inbound origin port.
Cloudflare Zero Trust Access	Applies identity-aware access rules and MFA requirements.
Gunicorn	Optional production WSGI server for hosting the Flask app.

5. Implementation Details

Application login

The login form accepts a username, password, and six-digit TOTP code. Credentials are compared using constant-time comparison where applicable. A generic error message avoids revealing which factor was incorrect.

Session handling

The session is cleared on successful login and uses HttpOnly and SameSite cookie settings. The Secure cookie flag can be enabled when deployment and proxy behavior have been verified.

Login throttling

A simple in-memory throttle blocks repeated login attempts from the same observed client address. It is intended for demonstration only; production deployments should use shared storage and edge/gateway rate limiting.

Cloudflare Access identity header

An optional presence check can require the configured Access identity header. Header presence alone is not cryptographic verification; if origin access can bypass Access, validate the Cf-Access-Jwt-Assertion signature, issuer, audience, and expiry.

6. Setup and Deployment Procedure

1. Create a Python virtual environment and install the packages listed in requirements.txt.

2. Set FLASK_SECRET_KEY, ADMIN_USERNAME, ADMIN_PASSWORD, and a securely provisioned Base32 MFA_SECRET in environment variables.

3. Run the Flask app locally and verify the login and dashboard using a valid TOTP code.

4. Create a named Cloudflare Tunnel, route a DNS hostname to it, and configure the ingress service to http://127.0.0.1:5000.

5. Create a Cloudflare Access self-hosted application for the hostname and restrict access to approved identities/groups.

6. Configure MFA requirements in the identity provider / Access policy, then test both allowed and denied users.

7. Confirm the origin is not directly exposed and keep tunnel credentials outside version control.

7. Testing Plan

Test case	Expected result
Correct username/password + valid TOTP	Login succeeds and dashboard is displayed.
Incorrect password or TOTP	Generic authentication error; dashboard remains protected.

Test case	Expected result
Repeated failed login attempts	HTTP 429 after the configured threshold within the time window.
Unauthenticated request to /dashboard	Redirect to login.
Identity not permitted by Access policy	Cloudflare Access denies request before it reaches the app.
Tunnel stopped / origin unavailable	Hostname does not reach the local application; verify expected error handling.

8. Security Considerations and Limitations

• The sample code is an academic prototype; it does not provide password hashing/database-backed user management, CSRF protection, account recovery, central audit logs, or a shared rate limiter.

• The application-level TOTP secret must be provisioned and stored securely. Do not put real secrets in source control.

• Cloudflare Tunnel does not, by itself, enforce user identity. Cloudflare Access application and policy configuration is required.

• Do not trust identity headers if direct access to the origin is possible. Validate Access JWTs at the origin for deployments that rely on those claims.

• Use a production WSGI server, dependency updates, monitoring, backups, secret rotation, incident response procedures, and a formal security review before real-world use.

9. Expected Outcome

When configured correctly, only identities allowed by Cloudflare Access can reach the application, and successful application login requires both a password and a valid TOTP code. The origin remains locally bound and is reached through the Tunnel rather than an exposed inbound application port. These controls reduce attack surface and demonstrate layered identity verification.

10. Conclusion

The project provides a practical introduction to zero-trust access for internal administrative interfaces. It combines network exposure reduction through Cloudflare Tunnel, identity-aware policy enforcement through Cloudflare Access, and application-level TOTP MFA. The code and configuration are intentionally educational; production use requires stronger identity lifecycle management, cryptographic Access-token validation where appropriate, centralized logging and throttling, CSRF defenses, and security testing.

References

• Cloudflare Zero Trust documentation: https://developers.cloudflare.com/cloudflare-one/ • Cloudflare Tunnel documentation: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/ • Flask documentation: https://flask.palletsprojects.com/ • PyOTP documentation: https://pyauth.github.io/pyotp/

Prepared as an educational project report. Configure and validate all access policies in the actual Cloudflare Zero Trust dashboard before deployment.